

Asynchronous JavaScript & RESTful APIs

Real-Time Weather Dashboard Solution & Reference Guide

PROJECT DOMAIN	API PROVIDER	TARGET OUTCOME	SUGGESTED REPO
Asynchronous JS & REST APIs	OpenWeatherMap REST API	Live Weather Dashboard	weather-dashboard-js

1. Project Overview & Requirements

Build a real-time Weather Dashboard by fetching and processing JSON data from a public REST API using modern asynchronous JavaScript.

Key Features Implemented:

- Asynchronous Data Fetching:** Utilizes modern `fetch()` API combined with `async/await` syntax.
- Comprehensive Error Handling:** Gracefully catches network failures, HTTP status errors (e.g., 404 City Not Found), and empty user inputs.
- Nested JSON Parsing:** Dynamically extracts nested objects from API responses (e.g., temperature, humidity, wind speed, weather description).
- Search Functionality:** Allows users to query weather details by entering any valid city name with support for button clicks and the "Enter" key.

2. Step-by-Step Implementation Code

Step 1: HTML Structure (`index.html`)

Defines the markup for the search controls, status display, and weather metric cards.

```
index.html

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>Weather Dashboard</title>
  <link rel="stylesheet" href="style.css" />
</head>
<body>
  <div class="card">
    <h2 class="title">Weather Dashboard</h2>

    <!-- Search Box -->
    <div class="search-box">
      <input type="text" id="cityInput" placeholder="Enter city name..." />
      <button id="searchBtn">Search</button>
    </div>

    <!-- Status & Error Display -->
```

```

<div id="statusMessage" class="status-msg"></div>

<!-- Weather Display Metrics -->
<div id="weatherDisplay" class="weather-info hidden">
  <h3 id="cityName"></h3>
  <div class="temp-box">
    <span id="temperature">--</span>°C
  </div>
  <p id="description" class="description">--</p>

  <div class="metrics-grid">
    <div class="metric">
      <span class="label">Humidity</span>
      <span id="humidity" class="value">--%</span>
    </div>
    <div class="metric">
      <span class="label">Wind Speed</span>
      <span id="windSpeed" class="value">-- m/s</span>
    </div>
  </div>
</div>
</div>

<script src="app.js"></script>
</body>
</html>

```

Step 2: CSS Styling (style.css)

Provides a responsive layout with metric panels, status indicators, and clean UI components.

style.css

```

* {
  box-sizing: border-box;
  margin: 0;
  padding: 0;
  font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}

body {
  background: #f0f4f8;
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh;
}

.card {
  background: #ffffff;
  padding: 2rem;
  border-radius: 12px;
  box-shadow: 0 10px 25px rgba(0, 0, 0, 0.08);
  width: 100%;
  max-width: 400px;
}

.title {
  text-align: center;
  color: #2c3e50;
}

```

```
margin-bottom: 1.5rem;
}

.search-box {
  display: flex;
  gap: 10px;
  margin-bottom: 1rem;
}

input {
  flex: 1;
  padding: 10px 14px;
  border: 1px solid #dcdfe6;
  border-radius: 6px;
  font-size: 1rem;
  outline: none;
}

button {
  padding: 10px 18px;
  background-color: #3b82f6;
  color: white;
  border: none;
  border-radius: 6px;
  font-size: 1rem;
  cursor: pointer;
}

button:hover {
  background-color: #2563eb;
}

.status-msg {
  text-align: center;
  margin-top: 10px;
  font-size: 0.95rem;
  color: #64748b;
}

.status-msg.error {
  color: #ef4444;
}

.hidden {
  display: none;
}

.weather-info {
  text-align: center;
  margin-top: 1.5rem;
}

.temp-box {
  font-size: 3rem;
  font-weight: bold;
  color: #1e293b;
  margin: 10px 0 5px;
}

.description {
  text-transform: capitalize;
```

```

    color: #64748b;
    margin-bottom: 1.5rem;
  }

.metrics-grid {
  display: flex;
  justify-content: space-around;
  background: #f8fafc;
  padding: 1rem;
  border-radius: 8px;
}

.metric {
  display: flex;
  flex-direction: column;
}

.metric .label {
  font-size: 0.85rem;
  color: #94a3b8;
}

.metric .value {
  font-size: 1.1rem;
  font-weight: 600;
  color: #334155;
}

```

Step 3: Asynchronous Logic (app.js)

Contains the core asynchronous network request logic, dynamic rendering, and error handling.

app.js

```

// Configuration
const API_KEY = "YOUR_API_KEY"; // Replace with OpenWeatherMap API key
const BASE_URL = "https://api.openweathermap.org/data/2.5/weather";

// DOM Elements
const searchBtn = document.getElementById("searchBtn");
const cityInput = document.getElementById("cityInput");
const statusMessage = document.getElementById("statusMessage");
const weatherDisplay = document.getElementById("weatherDisplay");

const cityNameEl = document.getElementById("cityName");
const temperatureEl = document.getElementById("temperature");
const descriptionEl = document.getElementById("description");
const humidityEl = document.getElementById("humidity");
const windSpeedEl = document.getElementById("windSpeed");

// 1. Asynchronous Fetch Request
async function fetchWeatherData(city) {
  try {
    setStatusMessage("Loading weather data...", false);
    weatherDisplay.classList.add("hidden");

    const response = await fetch(
      `${BASE_URL}?q=${encodeURIComponent(city)}&appid=${API_KEY}&units=metric`
    );
  }
}

```

```

    if (!response.ok) {
      if (response.status === 404) {
        throw new Error("City not found. Please verify input.");
      } else {
        throw new Error("Failed to retrieve weather data.");
      }
    }
  }

  const data = await response.json();
  renderWeatherData(data);
  setStatusMessage("", false);
} catch (error) {
  setStatusMessage(error.message, true);
}
}

// 2. Parsing Nested JSON & DOM Manipulation
function renderWeatherData(data) {
  const { name, sys, main, weather, wind } = data;

  cityNameEl.textContent = `${name}, ${sys.country}`;
  temperatureEl.textContent = Math.round(main.temp);
  descriptionEl.textContent = weather[0].description;
  humidityEl.textContent = `${main.humidity}%`;
  windSpeedEl.textContent = `${wind.speed} m/s`;

  weatherDisplay.classList.remove("hidden");
}

function setStatusMessage(msg, isError) {
  statusMessage.textContent = msg;
  statusMessage.className = isError ? "status-msg error" : "status-msg";
}

// 3. Event Handling
searchBtn.addEventListener("click", () => {
  const city = cityInput.value.trim();
  if (city) {
    fetchWeatherData(city);
  } else {
    setStatusMessage("Please enter a city name.", true);
  }
});

cityInput.addEventListener("keypress", (e) => {
  if (e.key === "Enter") searchBtn.click();
});

```

3. Key Concepts & Architecture Summary

Concept	Implementation Detail	Purpose
Fetch API	<code>fetch(url)</code>	Initiates an asynchronous HTTP request to the OpenWeatherMap REST API.
Async / Await	<code>async function & await response.json()</code>	Simplifies asynchronous execution flow, making code readable and synchronous-looking.

Concept	Implementation Detail	Purpose
Error Handling	<code>try...catch</code> + <code>response.ok</code> check	Catches network offline errors and HTTP errors like 404 (City Not Found).
JSON Parsing	<code>const { name, sys, main } = data</code>	Deconstructs deeply nested properties from JSON payloads to populate DOM elements.

Repository & Submission Guidelines

Recommended Repository Names for version control submission:

- `weather-dashboard-js` (*Recommended*)
- `async-js-weather-dashboard`
- `thiranex-weather-app`

Setup Steps: Initialize project with `index.html`, `style.css`, and `app.js`, register for an OpenWeatherMap API key, replace `YOUR_API_KEY`, and push to GitHub/GitLab.